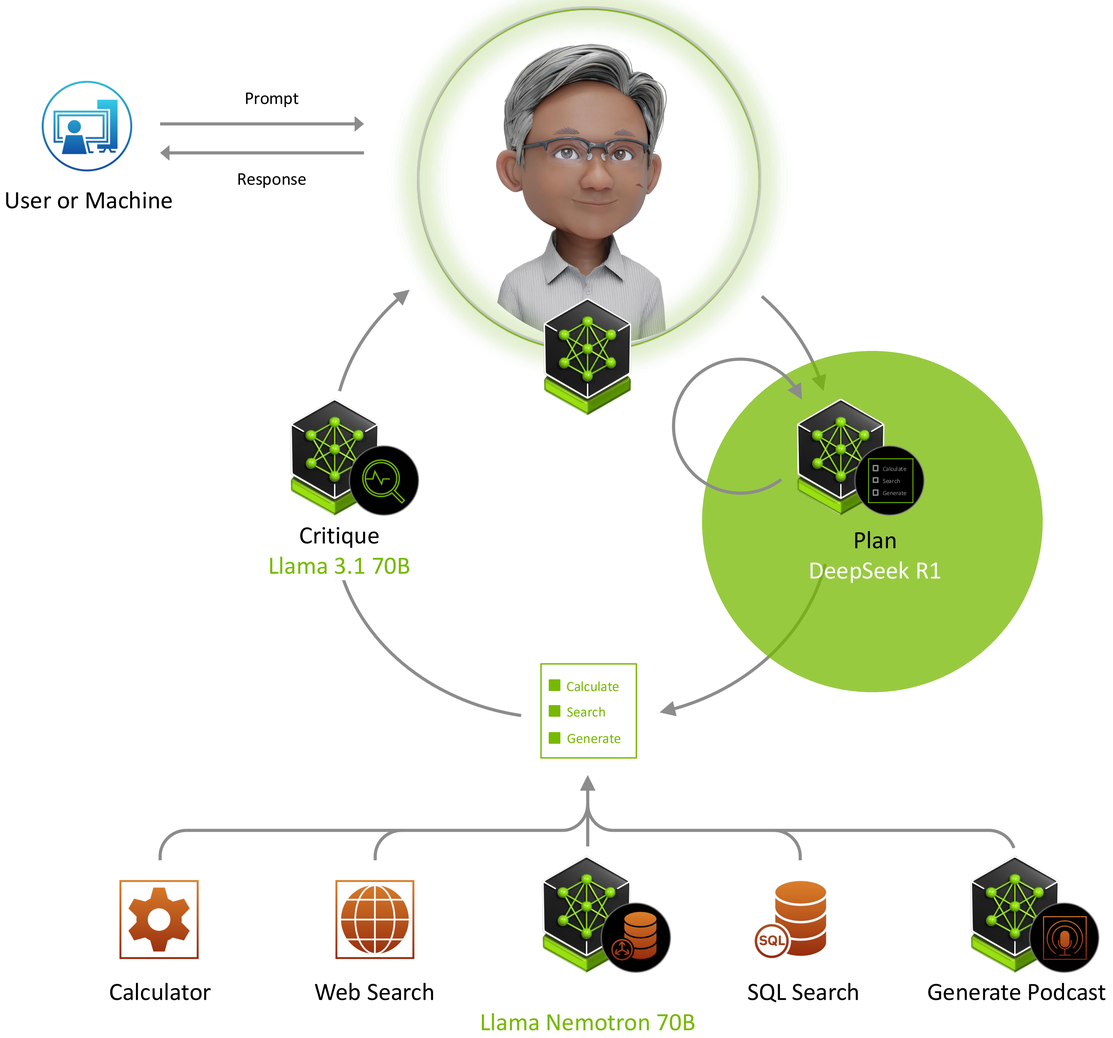
實作工作坊：快速上手、下載與建置
NVIDIA NIM

Cliff Chiu, Solution Architect | May 2026

AI Agents Require Multiple Models and Tools

Developers want to rapidly integrate the latest AI building blocks




```
import openai

  client = openai.OpenAI (

    base_url = “MY_LOCAL_ENDPOINT_URL”

  )

critique_completion = client.chat.completions.create (

  model = “llama-3.1-70b”

  )

plan_completion = client.chat.completions.create (

  model = “deepseek-r1” // model = “llama-3.1-405b”

  )
```

NVIDIA NIM Optimized Inference Microservices

Rapidly deploy reliable building blocks for accelerated generative AI anywhere

Portable Run cloud-native microservices anywhere, maintaining security and control of data and apps

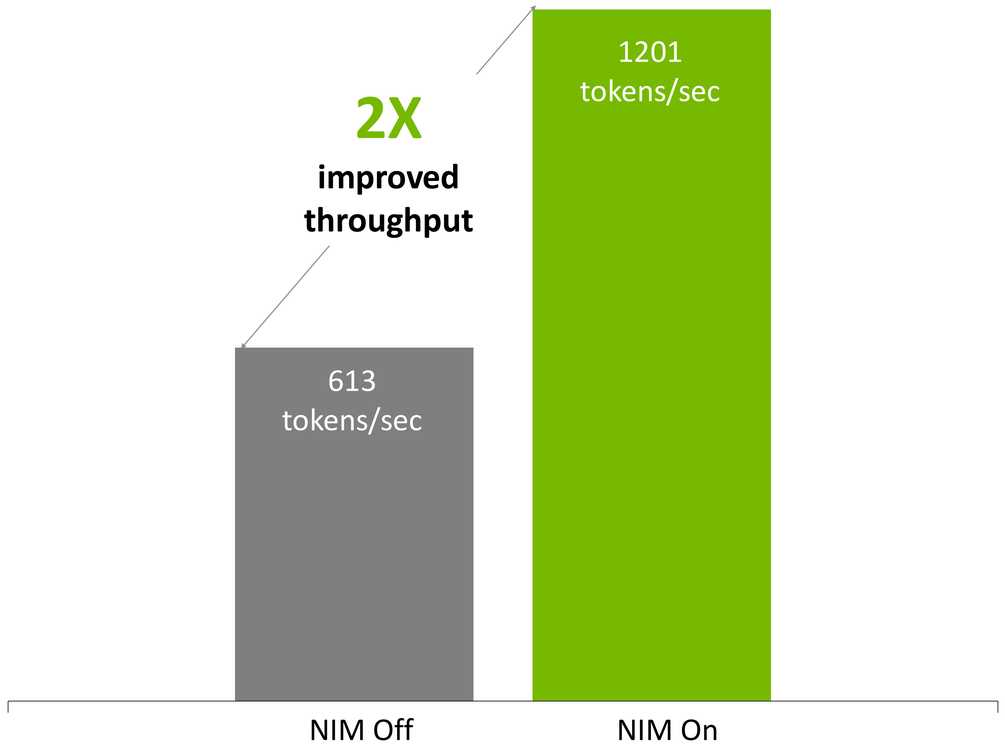
Easy to Use Move fast with the latest agentic AI building blocks for reasoning, retrieval, images and more, deployed in minutes with standard APIs

Enterprise Supported Gain confidence with stable APIs, quality assurance, continuous updates, security patching, and support

Performance Optimize accuracy, latency and throughput to meet requirements with lowest TCO

Optimized Efficiency Out of the Box

Improved performance on the same infrastructure with every release

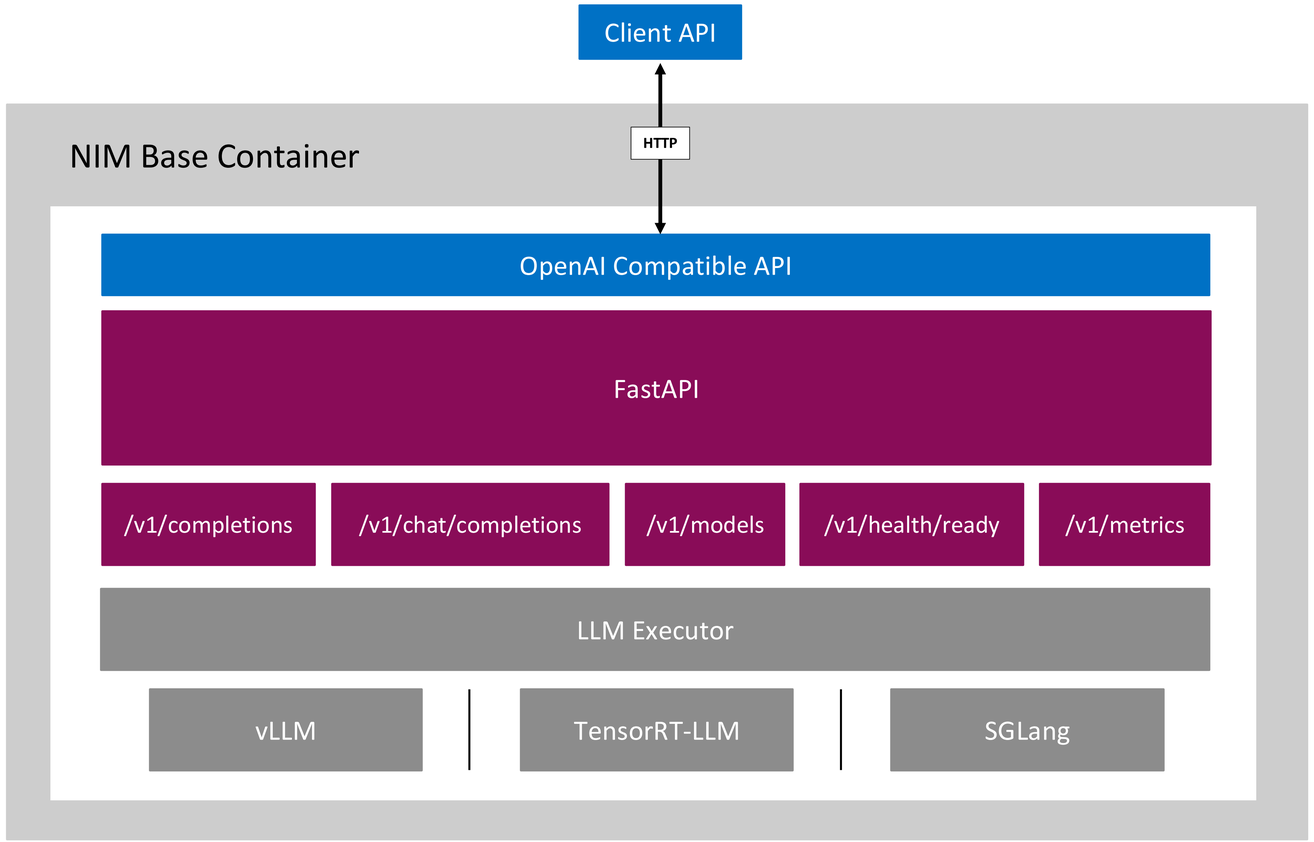


LLM NIM Benchmarking Guide

Llama 3.1 8B, 1 x H100SXM, concurrent requests: 200. NIM ON: NIM v1.3, FP8. throughput 1,201 tokens/s, ITL 32ms. NIM OFF: FP8. throughput 613 tokens/s, ITL 37ms.

NVIDIA NIM for LLM Architecture

- HTTP REST API conforms to OpenAI specification for easy developer integration
- Liveness, health check and metrics endpoints for monitoring and enterprise management
- NVIDIA NIM includes multiple LLM runtimes: TensorRT-LLM, vLLM and SGLang



NVIDIA NIM Powers Generative AI Factories

Building blocks of multi-modal agentic AI

Language & Reasoning

Regional and Domain Specialized

Visual

RAG

Speech

Digital Human

Healthcare

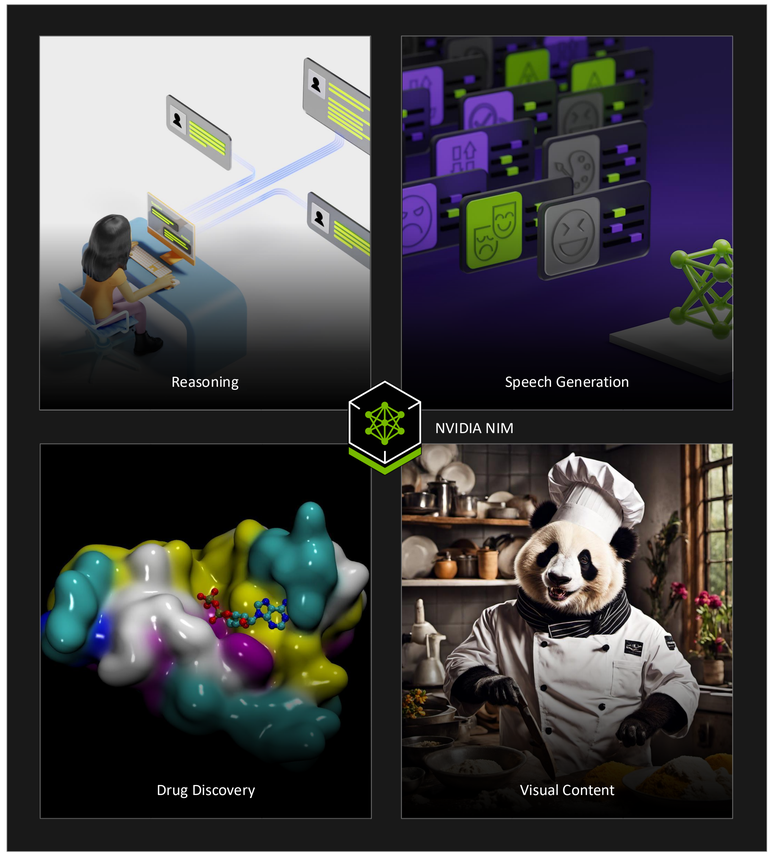
Computer Vision & Simulation

Safety

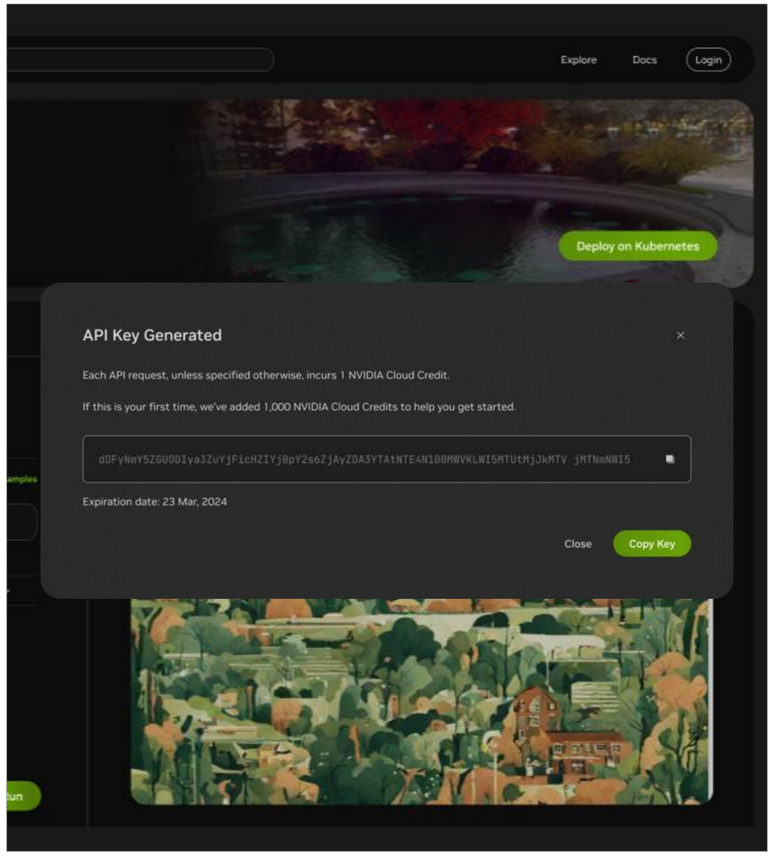
build.nvidia.com

Experience and Run Enterprise Generative AI Models Anywhere

Get started at build.nvidia.com



Experience Models



Prototype with APIs

Deploy with NIM

Hands-on Lab

實作工作坊大綱

Getting Started with NIM

- 範例一：LLM-NIM - Nemotron 3 A3B Nano
- 實作：使用 AIPerf 進行測試
- 課堂測驗：使用 AIPerf 進行測試

硬體環境需求

Getting Started with NIM

1. 中央處理器 CPU

- 建議 8 核心以上的 x86 處理器

2. 顯示卡 GPU

CUDA compute capability > 7.0

(B200, H200, H100, A100, RTX PRO 6000)

3. 系統記憶體與 GPU 記憶體 (host memory and GPU memory)

- 作業系統和其他程序需要 5-10 GB 內存
- Docker 16 GB 記憶體
- # 模型參數 * 2 GB 內存
  - Llama 8B：約 16 GB
  - Llama 70B：約 140 GB
  - 其他以此類推

事前準備工作

Getting Started with NIM

1. 準備環境

- NVIDIA Driver
- Docker Engine
- NVIDIA Container Toolkit

2. 獲取 API Key

前往 build.nvidia.com 註冊

並生成 NGC API Key。

3. 登入 Registry

使用 Docker Login 登入 nvcr.io

驗證權限

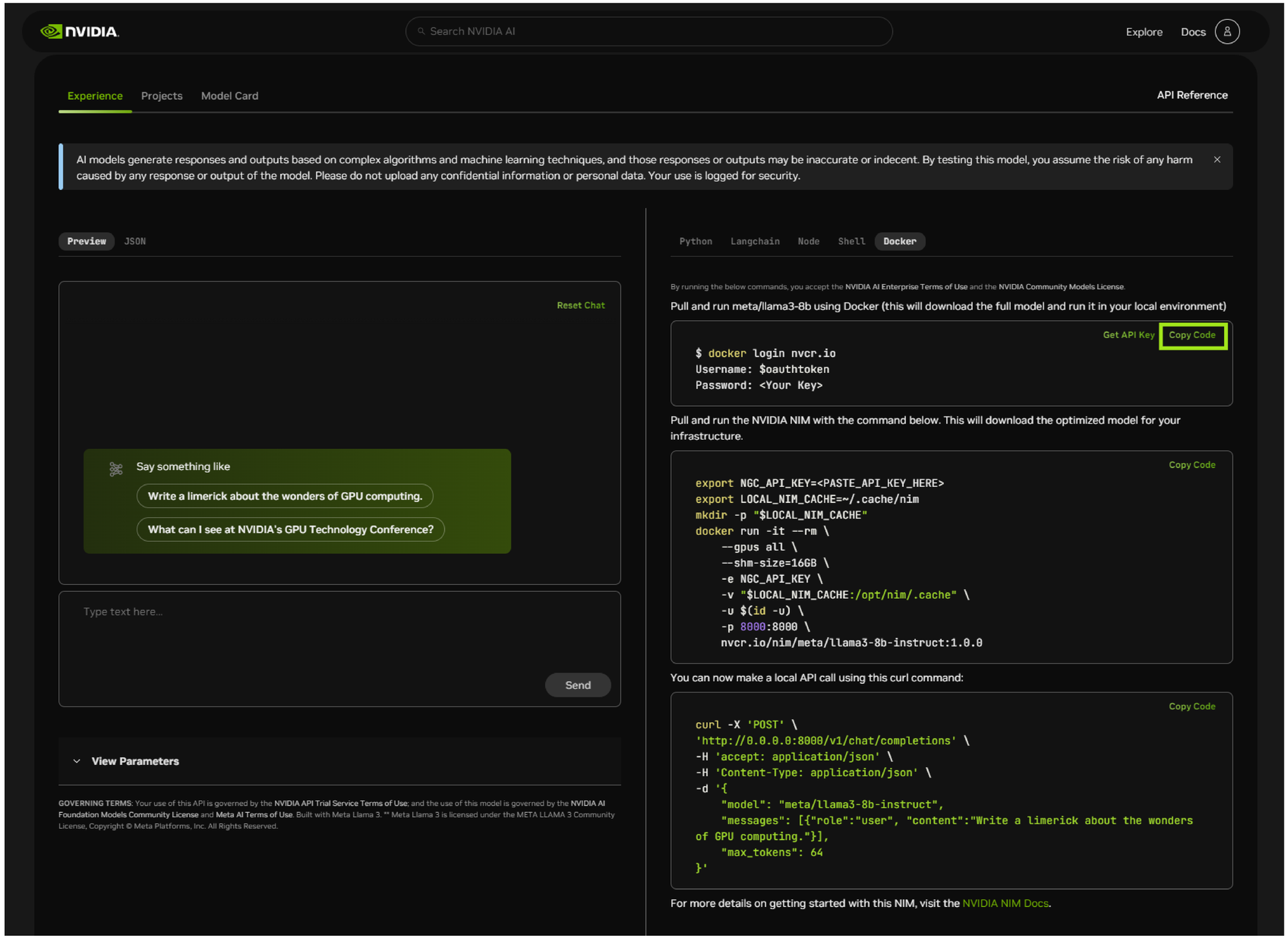
設定 API Key 並登入 Docker

```
export NGC_API_KEY=<your-api-key>
echo "$NGC_API_KEY" | docker login nvcr.io --username '$oauthtoken' --password-stdin
```

事前準備工作

前往 build.nvidia.com 註冊

獲取 NGC API Key



範例： LLM-NIM - Nemotron 3 A3B Nano (1/3)

啟動推論伺服器

啟動 Nemotron 3 A3B Nano 模型

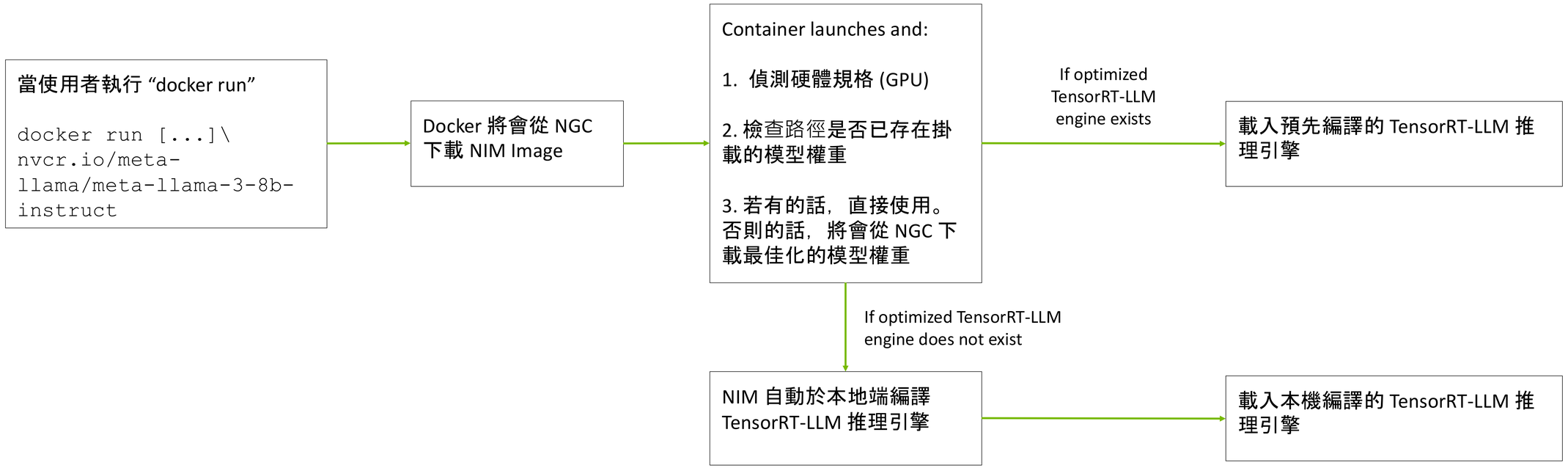
```
# Choose a path on your system to cache the downloaded models
export LOCAL_NIM_CACHE=~/.cache/nim

# Start the LLM NIM
docker run -it --rm \
    --gpus device=0 \
    --shm-size=16GB \
    -e NGC_API_KEY \
    -v "$LOCAL_NIM_CACHE:/opt/nim/.cache" \
    -p 8000:8000 \
    nvcr.io/nim/nvidia/nemotron-3-nano:latest \
    nim-serve \
    --enable-auto-tool-choice \
    --tool-call-parser qwen3_coder \
    --reasoning-parser nemotron_v3
```

- --gpus device=0 : 使用編號為 0 的 GPU
- --shm-size=16GB : 設定共享記憶體。
- -e NGC_API_KEY : 設定 API KEY 從 NGC 下載模型權重
- -v … : 掛載本機快取目錄，下次使用可重複利用模型。
- -p 8000:8000 : 開放 API 埠口。

NVIDIA LLM NIM 工作流程

LLM-specific NIM



範例一： LLM-NIM - Nemotron 3 A3B Nano (2/3)

發送請求

發送請求以獲取模型資訊

```
curl -X GET 'http://0.0.0.0:8000/v1/models'
```

發送 OpenAI Completion Request

```
curl -X 'POST' \
    'http://0.0.0.0:8000/v1/completions' \
    -H 'accept: application/json' \
    -H 'Content-Type: application/json' \
    -d '{
      "model": "nvidia/nemotron-3-nano",
      "prompt": "Once upon a time",
      "max_tokens": 64
    }'
```

發送 OpenAI Chat Completion Request

```
curl -X 'POST' \
'http://0.0.0.0:8000/v1/chat/completions' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
    "model": "nvidia/nemotron-3-nano",
    "messages": [{"role":"user", "content":"Which
number is larger, 9.11 or 9.8?"}],
    "max_tokens": 256
}'
```

範例一： LLM-NIM - Nemotron 3 A3B Nano (3/3)

進階功能

推理模式 Reasoning

新增額外參數 enable_thinking

```
curl -X 'POST' \
'http://0.0.0.0:8000/v1/chat/completions' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
    "model": "nvidia/nemotron-3-nano",
    "messages": [{"role":"user", "content":"Which
number is larger, 9.11 or 9.8?"}],
    "max_tokens": 256,
    "chat_template_kwargs": {
        "enable_thinking": true
     }
}'
```

硬體與模型配置

Hardware SKU & Model Profiles

NIM 會自動偵測硬體並選擇最佳設定，但在某些情況下（如多 GPU 配置或特定精度需求），我們需要手動選擇 Profile

如何查看 Model Profiles ?

列出所有可用的 Profiles

```
docker run --rm \
  --gpus device=0 \
  nvcr.io/nim/nvidia/nemotron-3-nano:latest \
  list-model-profiles
```

輸出將包含支援的 Profile 名稱與 hash value
(e.g., vllm-fp8-tp1-pp1, vllm-fp8-tp2-pp1)

選擇策略

Throughput
適合一般應用，選擇
Tensor Parallelism (TP) 較低的 Profile。

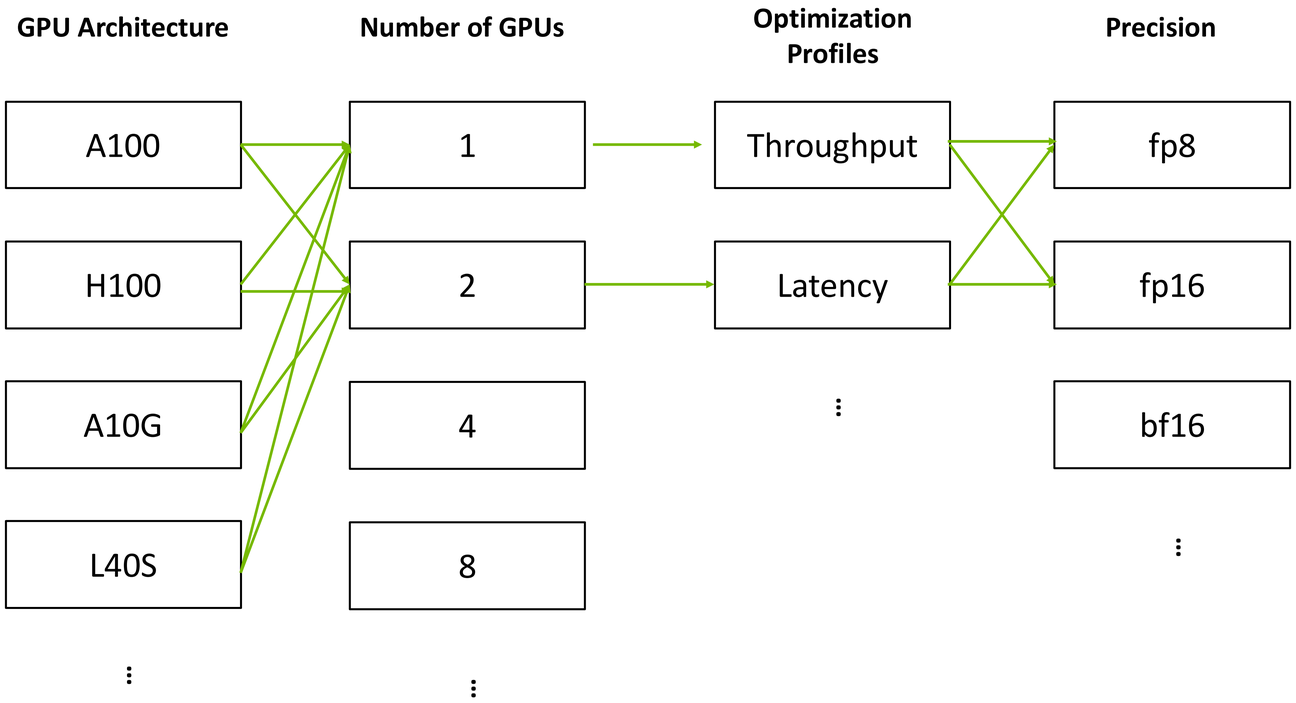
Latency (TTFT)
適合高負載的情境，選擇
Tensor Parallelism (TP) 較高的 Profile。

選擇特定的 Profile 啟動

```
docker run -it --rm \
    --gpus device=0 \
    --shm-size=16GB \
    -e NGC_API_KEY \
    -e
NIM_MODEL_PROFILE="352d88f8021d0ce396a61d10e929d1d4b45f75
038b593595d0d92f80a398a032" \
    -v "$LOCAL_NIM_CACHE:/opt/nim/.cache" \
    -p 8000:8000 \
    nvcr.io/nim/nvidia/nemotron-3-nano:latest
```

How NIMs Leverage Optimized Engines

最佳化推理引擎列表



Optimized Models

llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp8-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x2-fp8-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp16-**lora**

llama3-8b-instruct:0.10.0+cbc614f5-**a100**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x1-fp16-**lora**
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x2-fp16-**latency**

llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x1-fp16-**lora**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x2-fp8-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp8-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp16-**lora**

```
Detected 3 compatible profiles.
Valid profile: 17190a87573ad181e4a55b96d1a84b52f1c82c6542e2404473b25011ebfb403e (tensorrt_llm-A100-bf16-tp1-balanced)
Valid profile: 4ca08b7e7f1d59d5da5f761969320738d6922f25f26f334ab344c21f2e57348f (tensorrt_llm-A100-fp16-tp1-throughput)
Valid profile: 15fc17f889b55aedaccb1869bfeadd6cb540ab26a36c4a7056d0f7a983bb114f (vllm)
Selected profile: 17190a87573ad181e4a55b96d1a84b52f1c82c6542e2404473b25011ebfb403e (tensorrt_llm-A100-bf16-tp1-balanced)
Profile metadata: llm_engine: tensorrt-llm
```

離線部署

Air-gapped Deployment

在有網路的機器先下載所需模型權重與映像檔，再轉移至離線機器。

1. 在有網路的機器下載模型到快取資料夾 (download-to-cache)

```
# 設定 NGC API Key
export NGC_API_KEY=<your-api-key>
# 設定快取目錄
export NIM_CACHE_PATH=~/.cache/nim

# 執行下載指令
docker run -it --rm \
  -e NGC_API_KEY \
  -v $NIM_CACHE_PATH:/opt/nim/.cache \
  nvcr.io/nim/nvidia/nemotron-3-nano:latest \
  download-to-cache --profile
352d88f8021d0ce396a61d10e929d1d4b45f75038b593595d0d92f80a398a032
```

2. 在離線機器啟動 NIM

```
# 執行推理伺服器
docker run -it --rm \
    --gpus device=0 \
    --shm-size=16GB \
    -e NIM_MODEL_PROFILE="
352d88f8021d0ce396a61d10e929d1d4b45f75038b593595d0d92f80a398a0
32" \
    -v "$LOCAL_NIM_CACHE:/opt/nim/.cache" \
    -p 8000:8000 \
    nvcr.io/nim/nvidia/nemotron-3-nano:latest
```

效能基準測試指標

LLM Performance Metric

在部署 LLM 時，我們主要關注以下三個關鍵指標：

TTFT

Time To First Token

從發送請求到看到一個字出現的時間

影響使用者體驗 (回應延遲)

ITL

Inter-Token Latency

每個字生成之間的平均間隔時間

影響使用者流暢感 (生成速度)

Throughput

Requests / Tokens per Sec

系統每秒能處理的總請求數或字元數。

整體系統處理請求的能力

實作：使用 AIPerf 進行測試

Benchmarking

NVIDIA 提供了 **AIPerf** 工具來自動化產生多個請求並測量 LLM 推論相關效能指標。

Step 1 啟動執行 **AIPerf** 的環境，以 uv 為例

```
# Install uv
# curl -LsSf https://astral.sh/uv/install.sh | sh
# source "$HOME/.local/bin/env"

uv python install 3.12
uv venv aiperf --python 3.12
source aiperf/bin/activate
uv pip install aiperf
```

實作：使用 AIPerf 進行測試 (1/2)

Benchmarking

Step 3

```
aiperf profile \
    --model nvidia/nemotron-3-nano \
    --tokenizer nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-FP8 \
    --url http://0.0.0.0:8000 \
    --endpoint-type chat \
    --concurrency 128 \
    --request-count 384 \
    --streaming \
    --isl 128 \
    --isl-stddev 0 \
    --osl 512 \
    --osl-stddev 0 \
    --extra-inputs "ignore_eos:true" \
    --extra-inputs "min_tokens:512" \
    --random-seed 42 \
    --artifact-dir "artifacts"
```

- **--model nvidia/nemotron-3-nano**
  指定要進行效能測試的模型名稱。
- **--tokenizer nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-FP8**
  指定對應模型使用的 tokenizer，負責將文字轉換為模型可處理的 token。
- **--url http://0.0.0.0:8000**
  指定模型服務的 API 位址與連接埠。
- **--endpoint-type chat**
  指定使用 Chat（對話）型 API 介面進行請求。
- **--concurrency 128**
  同時發送的請求數量（併發數），用來模擬高併發使用情境。
- **--request-count 384**
  總共要發送的請求數量。
- **--isl 128**
  Input Sequence Length，指定每個請求的輸入 token 長度為 128。
- **--isl-stddev 0**
  輸入 token 長度的標準差為 0，表示所有請求輸入長度固定。
- **--osl 512**
  Output Sequence Length，指定模型最多生成 512 個 token。
- **--osl-stddev 0**
  輸出 token 長度的標準差為 0，表示輸出長度固定。
- **--extra-inputs "ignore_eos:true"**
  忽略 EOS（End of Sequence）標記，避免模型過早停止生成。
- **--extra-inputs "min_tokens:128"**
  指定模型至少要生成 128 個 token。
- **--random-seed 42**
  設定隨機種子，確保測試結果可重現。
- **--artifact-dir "/artifacts"**
  指定效能測試結果（log、metrics、報告）的輸出目錄。

實作：使用 AIPerf 進行測試 (2/2)

Benchmarking

NVIDIA AIPerf | LLM Metrics

Metric	avg	min	max	p99	p90	p50	std
Time to First Token (ms)	670.88	131.40	958.40	954.63	916.82	696.05	190.89
Time to Second Token (ms)	183.71	19.09	307.47	305.78	292.58	215.16	112.42
Time to First Output Token (ms)	9,309.73	3,062.30	18,953.36	18,676.87	14,673.82	8,384.62	3,620.34
Request Latency (ms)	19,578.91	19,436.00	19,687.04	19,668.29	19,640.88	19,580.56	47.74
Inter Token Latency (ms)	37.38	36.52	58.35	50.22	37.52	37.00	2.13
Output Token Throughput Per User (tokens/sec/user)	26.81	17.14	27.38	27.36	27.30	27.02	1.10
Output Sequence Length (tokens)	507.97	322.00	512.00	512.00	512.00	511.00	20.47
Input Sequence Length (tokens)	128.00	128.00	128.00	128.00	128.00	128.00	0.00
Output Token Throughput (tokens/sec)	3,308.52	N/A	N/A	N/A	N/A	N/A	N/A
Request Throughput (requests/sec)	6.51	N/A	N/A	N/A	N/A	N/A	N/A
Request Count (requests)	384.00	N/A	N/A	N/A	N/A	N/A	N/A